

Use Case Descriptions GUI

1. Previous Page
2. Login
3. Sign Up
4. Forgot Password
5. Home Page
6. Manage Profile
7. Edit Profile
8. Reset Password
9. Delete Profile
10. Logout
11. Add Friend
12. Search
13. Play Games
14. Select Tic Tac Toe
15. Select Connect 4
16. Select Checkers
17. Choose Opponent
18. Friend
19. Random
20. In-Game Options
21. Game Chat
22. Top Players
23. Game History
24. Quit

Use case: Previous Page

Iteration: 1

Primary Actor: Player

Goal in Context: The player wants to navigate to the previous page in the application.

Preconditions:

1. The player must be logged into the system or interacting with the application.
2. The system must have a history of navigated pages.

Trigger: The player clicks on the "**Previous Page**" button.

Scenario:

- The player initiates the action by selecting the "Previous Page" option.
- The system verifies if there is a previously visited page.
- If a previous page exists, the system loads it.
- The player is redirected to the previous page with the same state as when they left.

Postconditions:

- The player is successfully redirected to the previous page.
- Any unsaved data from the current page may be lost unless saved before navigation.

Exceptions:

- E1: If there is no previous page to navigate to, the system may display an error or disable the option.
- E2: If the previous page requires authentication and the session has expired, the system prompts the player to log in again.
- E3: If the previous page is no longer available (due to logout or session timeout), the system redirects the player to a default page (the home page).

Priority: Medium

When Available: This feature is available anytime a user has navigated away from an initial page.

Frequency of Use: Frequently used, as players often navigate between pages.

Channel to Actor: GUI, typically via a button or browser back function.

Use Case: Login

Iteration: 1

Primary Actor: Player

Goal in Context: *The player wants to authenticate and access their account in the system.*

Preconditions:

- The player must have an existing account or the option to sign up.
- The system must be available and responsive.
- The player must have a valid username/email and password.

Trigger:

- The player selects the **"Login"** option from the application interface.

Scenario (Main Flow):

- The player selects the **"Login"** option.
- The system prompts the player to enter their credentials (username/email and password).
- The player inputs their login details and submits the request.
- The system validates the credentials:
 - If valid, the system grants access and redirects the player to the **Home Page**.
 - If invalid, the system displays an error message.
- The player successfully accesses their account.

Postconditions:

- The player is authenticated and redirected to the **Home Page**.
- The session is started, allowing access to other features like playing games, managing profile, and adding friends.

Exceptions:

- **E1:** If the player enters incorrect credentials, the system displays an error and allows retrying.
- **E2:** If the player forgets their password, they can select **"Forgot Password"** to reset it.
- **E3:** If the system is down or experiencing issues, an error message is displayed, and the player is asked to try again later.
- **E4:** If the account is locked due to multiple failed attempts, the player is prompted to reset their password or contact support.

Priority:

High

When Available:

- Available at all times unless the system is undergoing maintenance.

Frequency of Use:

- Frequently used, as players need to log in to access their accounts.

Channel to Actor:

- **Graphical User Interface (GUI)** – via a login form.

Use Case: Sign Up

Iteration: 1

Primary Actor: Player

Goal in Context:

The player wants to create a new account to access the system's features, such as playing games and managing their profile.

Preconditions:

- The player must not already have an account.
- The system must be available and able to register new users.

Trigger:

- The player selects the "**Sign Up**" option on the login screen.

Scenario (Main Flow):

1. The player selects the "**Sign Up**" option.
2. The system displays a registration form requesting:
 - a. Username
 - b. Email address
 - c. Password (and confirmation)
 - d. Optional profile details
3. The player enters their details and submits the form.
4. The system validates the input:
 - a. Ensures all required fields are filled.
 - b. Checks if the email/username is already in use.
 - c. Verifies password strength.
5. If all inputs are valid, the system creates the account and sends a confirmation email (if required).
6. The player receives a confirmation message and is redirected to the login page.

Postconditions:

- The player's account is successfully created.
- The player can now log in and access the system.

Exceptions:

- **E1:** If required fields are missing, the system prompts the player to complete them.
- **E2:** If the email or username is already taken, the system notifies the player and suggests alternatives.

- **E3:** If the password does not meet security requirements, an error message is shown.
- **E4:** If system issues prevent account creation, an error is displayed, and the player is asked to try again later.

Priority:

High

When Available:

- Always available unless account registration is disabled for maintenance or other reasons.

Frequency of Use:

- Used whenever a new player wants to join the system.

Channel to Actor:

- **Graphical User Interface (GUI)** – via a registration form.

Use Case: Forgot Password

Iteration: 1

Primary Actor: Player

Goal in Context:

The player wants to reset their password to regain access to their account.

Preconditions:

- The player must have a registered account.
- The system must support password recovery.
- The player must have access to the registered email or other recovery methods.

Trigger:

- The player selects the "**Forgot Password**" option on the login screen.

Scenario (Main Flow):

1. The player clicks on "**Forgot Password**".
2. The system prompts the player to enter their registered email address.
3. The player enters their email and submits the request.
4. The system checks if the email exists in the database:
 - a. If found, the system sends a password reset link to the email.
 - b. If not found, the system displays an error message.
5. The player receives the password reset email and clicks on the provided link.
6. The system opens a reset password page where the player enters and confirms a new password.
7. The player submits the new password, and the system updates their credentials.
8. The system confirms the password reset and redirects the player to the **Login Page**.

Postconditions:

- The player's password is updated successfully.
- The player can now log in using the new password.

Exceptions:

- **E1:** If the email is not registered, the system informs the player that the email does not exist.
- **E2:** If the player does not receive the reset email, they may request it again after a certain period.
- **E3:** If the password reset link expires, the player must request a new one.
- **E4:** If the player enters mismatched passwords in the reset form, the system prompts them to re-enter the passwords.

- **E5:** If system issues prevent password reset, an error message is displayed, and the player is asked to try again later.

Priority:

High

When Available:

- Always available unless the system is undergoing maintenance.

Frequency of Use:

- Occasionally used when a player forgets their password.

Channel to Actor:

- **Graphical User Interface (GUI)** – via a form on the website or app.
- **Email System** – for sending the reset link.

Use Case: Manage Profile

Iteration: 1

Primary Actor: Player

Goal in Context:

The player wants to manage their account details, including updating personal information, changing the password, and deleting their profile.

Preconditions:

- The player must be logged into their account.
- The system must allow profile modifications.

Trigger:

- The player selects the **"Manage Profile"** option from the home page or settings menu.

Scenario (Main Flow):

1. The player selects **"Manage Profile"**.
2. The system displays profile management options, including:
 - a. **Edit Profile:** Modify username, email, or other personal details.
 - b. **Change Password:** Update login credentials.
 - c. **Delete Profile:** Permanently remove the account.
3. The player selects an option and provides the necessary input (e.g., new email, new password).
4. The system validates the input:
 - a. Ensures email is unique.
 - b. Checks password strength (if updating).
 - c. Confirms identity for sensitive actions (e.g., deleting the profile).
5. The system saves the changes and notifies the player.
6. The player sees a confirmation message.

Postconditions:

- The profile is updated with the new details.
- If the profile is deleted, the account is removed from the system.

Exceptions:

- **E1:** If the player enters an invalid or already-taken email, the system prompts them to try again.
- **E2:** If the password update does not meet security requirements, an error is displayed.

- **E3:** If the player attempts to delete their profile but cancels, no changes occur.
- **E4:** If system issues prevent updates, an error message is shown, and the player is asked to try again later.

Priority:

Medium

When Available:

- Always available when the player is logged in.

Frequency of Use:

- Occasionally used when players need to update their information.

Channel to Actor:

- **Graphical User Interface (GUI)** – via profile settings.

Use case: Home Page

Iteration: 1

Primary Actor: Player

Goal in context: Navigate to home page where games and menu options are available.

Preconditions:

1. The player has a registered account.
2. The player is successfully logged in.

Trigger: The player logs into the game platform successfully.

Scenario:

1. The player launches the game platform.
2. The login page is displayed.
3. The player enters valid credentials and logs in.
4. The player is redirected to the home page.
5. The home page displays the list of available games, added friend list, and a menu.

Post conditions:

1. The [player successfully sees the home page and can interact with it.
2. The player has access to all available features in the home page.

Exceptions:

1. The player enters incorrect credentials or network failure occurs and isn't able to login.
2. Home page does not load due to network failure.

Priority: High, the home page is where all features are accessible from.

When available: while logged in.

Frequency of use: Every time a game is picked, or any menu option is accessed.

Channel to actor: The games GUI

Use case: Logout

Iteration: 1

Primary Actor: Player

Goal in context: The player intends to terminate their session in the game system while ensuring that their personal data is preserved and protected.

Preconditions:

1. The player is logged into the game system.
2. The game system is operational.

Trigger: The player selects the log out option from the game menu or settings

Scenario:

1. The player navigates to the game platforms menu and selects the logout option.
2. The system asks the player to confirm that they wish to logout and the player confirms.
3. The system terminates the session, and the player is directed to the homepage.

Post conditions:

1. The session is terminated.
2. Temporary Data is cleared.

Exceptions:

1. Network Failure: player isn't able to properly logout if a network error occurs, process is stalled until issue is resolved.

Priority: High

When available: Available while player is logged in.

Frequency of use: High usage, mainly at the end of gaming sessions.

Channel to actor: The games GUI

Use case: Add Friend

Iteration: 1

Primary Actor: player

Goal in context: The player wants to add a friend.

Preconditions:

1. The player is logged into the system.
2. The system is operational.

Trigger: Player selects the add friend option from the menu.

Scenario:

1. The player sends a request to add a friend.
2. Upon confirmation, their friend is successfully added.

Post conditions:

1. The friend is added to player's list of friends.

Exceptions:

1. The friend does not accept the add request.
2. Network errors prevent request from being sent.

Priority: Medium

When available: Always when logged in and on the main page.

Frequency of use: Medium

Channel to actor: The game systems GUI

Use case: Search Friend

Iteration: 1

Primary Actor: Player

Goal in context: Search for a friend in the game system based on a unique user ID.

Preconditions:

1. Player must be logged in
2. Player must navigate to menu and the add friend option.

Trigger: Player selects the add friend option.

Scenario:

1. The player types in their friend's user ID.
2. If it exists, the system will display the friend's profile.

Post conditions:

1. The friend's profile has been successfully found.

Exceptions:

1. The user ID does not exist.
2. Network errors prevent the system from finding the profile.

Priority: Medium

When available: When logged into the game system and on the main page

Frequency of use: Medium

Channel to actor: The game systems GUI

Use case: Play Games

Iteration: 1

Primary Actor: Player

Goal in context: Select and join a game from the game menu to play.

Preconditions:

1. User is logged in.
2. User has navigated to the play games menu from the home page.

Trigger: The player has clicked the play game option from the home page.

Scenario:

1. The player selects the play game menu.
2. The System displays a list of available games.
3. The player selects a game to play.
4. The player can choose who to play with.
5. The player starts playing the selected game.

Post conditions:

1. The player successfully joined the selected game.

Exceptions:

1. Network failure prevents the player from seeing the list of games or joining one.

Priority: High

When available: While the player is logged in a in the game menu section.

Frequency of use: Whenever the player wants to play any game.

Channel to actor: The games GUI.

Use Case: Select Tic-Tac-Toe

Iteration: 1

Primary Actor: Player

Goal in Context: To initiate a match, the player selects Tic-Tac-Toe from the available games on the “**Play Games**” menu.

Preconditions:

1. The player is logged into the system.
2. The player has navigated to the “Play Games” section.
3. The system is online and operational.

Trigger:

1. The player interacts with the game selection menu and chooses Tic-Tac-Toe.

Scenario:

1. The player logs into the system and reaches the home page.
2. The player navigates to the “**Play Games**” section.
3. The system displays available game options (Tic-Tac-Toe, Checkers, Connect 4).
4. The player selects Tic-Tac-Toe.
5. The system includes the “Choose Opponent” use case.
6. The player selects an opponent (either a friend or a random player).
7. The system loads the Tic-Tac-Toe game interface.
8. The player now has access to in-game options, such as game chat, top players, game history, or quitting.

Postconditions:

1. The Tic-Tac-Toe game screen is displayed.
2. The player is matched with an opponent or waiting for one.
3. The game is ready to begin.

Exceptions:

- 1. Server Downtime:** If the game server is down, an error message is displayed.
- 2. Opponent Not Found:** If no opponent is available, the system prompts the player to wait or try again later.
- 3. Network Issues:** If the player loses connection, the system provides a reconnect option.

Priority: High

When Available: Initial game launch

Frequency of Use: Frequently, whenever a player wants to play Tic-Tac-Toe

Channel to Actor: Graphical User Interface (GUI)

Use case: Select Connect 4

Iteration: 1

Primary Actor: Player

Goal in context: The player selects Connect 4 from the available games in the “**Play Games**” menu to initiate a match.

Preconditions:

1. The player is logged into the system.
2. The player has accessed the “**Play Games**” section.
3. The system is online and functioning properly.

Trigger:

1. The player interacts with the game selection menu and chooses Connect 4.

Scenario:

1. The player logs into the system and reaches the home page.
2. The player navigates to the “**Play Games**” section.
3. The system displays available game options (Tic-Tac-Toe, Checkers, Connect 4).
4. The player selects Connect 4.
5. The system includes the “Choose Opponent” use case.
6. The player selects an opponent (either a friend or a random player).
7. The system loads the Connect 4 game interface.
8. The player now has access to in-game options, such as game chat, top players, game history, or quitting.

Post conditions:

1. The Connect 4 game screen is displayed.
2. The player is matched with an opponent or waiting for one.
3. The game is ready to begin.

Exceptions:

1. **Server Downtime:** If the game server is down, an error message is displayed
2. **Network Issues:** If the player loses connection, the system provides a reconnect option.
3. **Opponent Not Found:** If no opponent is available, the system prompts the player to wait or try again later.

Priority: High

When available: Initial game launch

Frequency of use: Frequently, whenever a player wants to play Connect 4

Channel to Actor: Graphical User Interface (GUI).

Use case: Select Checkers

Iteration: 1

Primary Actor: Player

Goal in context: The player selects Checkers from the available games in the “**Play Games**” menu to initiate a match.

Preconditions:

1. The player is logged into the system.
2. The player has accessed the “**Play Games**” section.
3. The system is online and functioning properly.

Trigger:

1. The player interacts with the game selection menu and chooses Checkers.

Scenario:

1. The player logs into the system and reaches the home page.
2. The player navigates to the “**Play Games**” section.
3. The system displays available game options (Tic-Tac-Toe, Checkers, Connect 4).
4. The player selects Checkers.
5. The system includes the “Choose Opponent” use case.
6. The player selects an opponent (either a friend or a random player).
7. The system loads the Checkers game interface.
8. The player now has access to in-game options, such as game chat, top players, game history, or quitting.

Post conditions:

1. The Checkers game screen is displayed.
2. The player is matched with an opponent or waiting for one.
3. The game is ready to begin.

Exceptions:

1. **Server Downtime:** If the game server is down, an error message is displayed
2. **Network Issues:** If the player loses connection, the system provides a reconnect option.
3. **Opponent Not Found:** If no opponent is available, the system prompts the player to wait or try again later.

Priority: High

When available: Initial game launch

Frequency of use: Frequently, whenever a player wants to play Checkers.

Channel to Actor: Graphical User Interface (GUI).

Use case: Choose Opponent

Iteration: 1

Primary Actor: Player

Goal in context: The player accesses the “**Choose Opponent**” page, where they can select to play against a friend or be matched with a random opponent.

Preconditions:

1. The player is logged in.
2. The player has selected a game (Tic-Tac-Toe, Connect 4, or Checkers).

Trigger:

1. The player has selected a game (Tic-Tac-Toe, Connect 4, or Checkers).

Scenario:

1. The player selects a game from the available options.
2. The system presents the “Choose Opponent” option.
3. The player clicks on “Choose Opponent.”
4. The system redirects the player to the opponent selection page, where they can choose between a friend or a random opponent

Post conditions:

1. The player reaches the opponent selection page.

Exceptions:

1. If the player takes too long, they may need to restart the selection process

Priority: High

When available: Initial game launch.

Frequency of use: Every time a player initiates a game session.

Channel to actor: Graphical User Interface (GUI).

Use case: Friend (Opponent Selection)

Iteration: 1

Primary Actor: Player

Goal in context: The player selects a friend as their opponent to initiate a game session.

Preconditions:

1. The player is logged into their account.
2. The player has selected a game (Tic-Tac-Toe, Connect 4, or Checkers).
3. The player has reached the opponent selection page.
4. The player has at least one friend added to their list.

Trigger:

1. The player chooses the “**Friend**” option from the opponent selection page.

Scenario:

1. The player selects a game.
2. The system navigates the player to the “**Choose Opponent**” page.
3. The player selects the “**Friend**” option.
4. The system displays a list of available friends.
5. The player selects a friend from the list.
6. The system sends an invitation to the selected friend.
7. If the friend accepts, the game session starts.

Post conditions:

1. If the friend accepts, the game begins.
2. If the friend declines or does not respond, the player can choose another friend or return to the opponent selection page.

Exceptions:

1. **No Friends Available:** If the player has no friends added, the system displays a message and suggests adding friends.
2. **Friend is Unavailable:** If the selected friend is offline or in another game, an error message is shown.
3. **Invitation Timeout:** If the friend does not respond within a certain time, the system cancels the request.

Priority: High

When available: Initial game launch

Frequency of use: Whenever a player wants to play against a friend.

Channel to actor: Graphical User Interface (GUI).

Use case: In-Game Options

Iteration: 1

Primary Actor: Player

Goal in context: To allow the player to utilize various in-game functionalities, including game chat, top players, game history, and game termination.

Preconditions:

1. The player must be logged into their account.
2. The player must have started a game.

Trigger: Once the opponent is selected, the game starts, and in-game options become available for interaction on different parts of the screen.

Scenario:

1. The player selects an opponent (either a friend or a random player).
2. The system starts the game session and displays the game board.
3. In-game options (chat, top players, game history, quit) appear in designated areas of the screen.
4. The player interacts with an option by selecting its corresponding button on the screen.
5. The system executes the selected option and presents it as a pop-up window.

Post conditions:

1. Player is allowed to select a desired option and appropriate window appears.

Exceptions:

1. If the game is disconnected due to a network issue, in-game options may not function and show connection error.
2. Some options (e.g., game history, leaderboard) might be unavailable due to database error and shows data could not be found.

Priority: High, gives the user option to exit a game and talk to opponent.

When available: After selecting an opponent and the game being initiated.

Frequency of use: Frequently used during gameplay.

Channel to actor: The game GUI

Use case: Game Chat

Iteration: 1

Primary Actor: Player

Goal in context: Allow the player to communicate with their opponent in real-time during a game.

Preconditions:

1. The player must be logged into their account.
2. The player must have started a game and has an opponent
3. The game session must be active.

Trigger: Once the Game Chat is selected from the displayed option on the board.

Scenario:

1. The player starts a game after selecting an opponent.
2. The system displays the game board along with in-game options.
3. The player selects the Game Chat option from the screen.
4. A chat window opens, allowing the player to send and receive messages.
5. The player types a message and sends it.
6. The system delivers the message to the opponent in real-time.
7. The opponent can respond, and the conversation continues throughout the active game session.

Post conditions:

1. The player's messages are sent and received successfully.
2. The chat interaction remains available during the game session.

Exceptions:

1. If the network connection is lost, chat messages may fail to send or be delayed.
2. If the opponent exits the game, chat functionality may become disabled.
3. If chat moderation rules apply, certain messages may be filtered or blocked.

Priority: Moderate

When available: After selecting an opponent and starting a game.

Frequency of use: Frequently used during gameplay.

Channel to actor: The game GUI with the chat window pop-up

Use case: Top Players

Iteration: 1

Primary Actor: Player

Goal in context: Allow the player to view a leaderboard displaying the highest-ranked players based on game performance.

Preconditions:

1. The player must be logged into their account.
2. The player must have started a game and has an opponent
3. The leaderboard data must be available in the system.

Trigger: Once the Top Players is selected from the displayed option on the board.

Scenario:

1. The player starts a game after selecting an opponent.
2. The system displays the game board along with in-game options.
3. The player selects the Top Players option from the screen.
4. The system retrieves the leaderboard data from the server.
5. The system displays a ranked list of top players, showing details such as usernames, scores, and win rates. (without interrupting the state of game)
6. The player can scroll through the list to view rankings and can continue the as desired.

Post conditions:

1. The leaderboard is displayed successfully.
2. The player can view the rankings without interrupting the game session.

Exceptions:

1. If there is a network issue, the leaderboard may fail to load.
2. If no ranking data is available, the system might have database error and shows data could not be found.

Priority: Medium

When available: After selecting an opponent and starting a game.

Frequency of use: Occasionally, depending on player's interest in Ranks.

Channel to actor: The game GUI with Ranking Pop-up

Use case: Game History

Iteration: 1

Primary Actor: Player

Goal in context: Allow the player to view stats of previously played games.

Preconditions:

1. The player must be logged into their account.
2. The player must have completed at least one game.
3. Game history data must be stored and accessible in the system.

Trigger: Once the Game History is selected from the displayed option on the board.

Scenario:

1. The player starts a game after selecting an opponent.
2. The system displays the game board along with in-game options.
3. The player selects the Game History option from the screen.
4. The system retrieves and displays a list of previously played games.
5. The player can view details such as opponents, win stats, and timestamps.
6. The player can close the history panel and continue playing.

Post conditions:

1. The game history is displayed successfully.
2. The player can review past matches without interrupting the current session.

Exceptions:

1. If there is a network issue, game history data may not load.
2. If no game history is available, the system may display an error and show No records available.

Priority: Medium

When available: After selecting an opponent and starting a game.

Frequency of use: Occasionally, depending on player's interest in reviewing past gameplay stats.

Channel to actor: The game GUI with Game History Pop-up.

Use case: Quit

Iteration: 1

Primary Actor: Player

Goal in context: The goal of this use case is to allow the player to safely exit the online multiplayer board game platform at any point during their session.

Preconditions:

1. The player must be logged into the system.

Trigger: The player selects the "Quit" option from the main.

Scenario:

1. The player selects the "Quit" option.
2. The system prompts the player to confirm their intention to quit.
3. Upon confirmation, the system performs any necessary cleanup tasks, like saving game progress and logging out the user.
4. The system closes the application window or returns the player to the initial login screen.
5. The system logs the action of quitting for session analysis.

Post conditions:

1. The player's current session is safely terminated without loss of data.
2. The player is logged out of the system.

Exceptions:

1. If the player cancels the quit operation, they are returned to their last active screen within the system.

Priority: High

When available: Available throughout the player's session, accessible from any point within the platform's user interface.

Frequency of use: Multiple times per session.

Channel to actor: The games GUI

Use case: Reset Password

Iteration: 1

Primary Actor: Player

Goal in context: Allow players to securely reset their password if they have forgotten it or wish to change it.

Preconditions:

1. The player must have an existing account with the system.
2. The player has access to the email associated with their account.

Trigger: The player selects the "Forgot Password" or "Reset Password" option on the login screen or within their account settings.

Scenario:

1. The player selects the "Reset Password" option.
2. The system prompts the player to enter their registered email address or username.
3. The system verifies the provided information and sends a password code to the player.
4. The player enters the code.
5. The system prompts the player to enter the new password and confirm it.
6. The player submits the new password.
7. The system updates the account with the new password and confirms the successful password reset to the player.
8. The player can now log in with the new password.

Post conditions:

1. The player's password is updated in the system.
2. The player can access their account using the new password.

Exceptions:

1. If the player enters an email address or username not associated with any account, the system notifies them of the error.
2. If the player's email or mobile number is not accessible, they cannot complete the reset process and may need to contact support.
3. If the password code expires or is invalid, the player must request a new password reset.

Priority: High

When available: Always

Frequency of use: Not Frequent

Channel to actor: The Games GUI

Use case: Delete Profile

Iteration: 1

Primary Actor: Player

Goal in context: The user wants to permanently delete their profile from the system, including all associated data.

Preconditions:

3. The user must be logged into their account.

Trigger: The player initiates the profile deletion process.

Scenario:

1. The player navigates to the delete account option.
2. The player selects the “Delete Profile” option.
3. The system prompts the player to confirm the deletion.
4. The player provides confirmation.
5. The system validates the request and deletes the player’s profile and data.
6. The system validates the request and deletes the player’s profile and data.
7. The player is logged out, and their account is no longer accessible.

Post conditions:

1. The player’s profile and all associated data are permanently removed from the system.
2. The user is logged out and cannot log in again.

Exceptions:

1. If a system failure occurs, the deletion request is logged, and the player is asked to try again later.

Priority: High

When available: Always available from “Manage Profile”.

Frequency of use: Rare

Channel to actor: The games GUI

Use case: Edit Profile

Iteration: 1

Primary Actor: Player

Goal in context: The player wants to update their profile information.

Preconditions:

1. The player must be logged into their account.

Trigger: The player selects the “Edit Profile” option from the “Manage Profile” page.

Scenario:

1. The player navigates to the “Manage Profile” page.
2. The player selects the “Edit Profile” option.
3. The system displays the editable profile fields.
4. The player updates the desired information.
5. The player submits the changes by clicking the “Save” button.
6. The system validates the input data.
7. If the data is valid, the system updates the profile and confirms the changes.
8. The updated information is reflected in the player’s profile.

Post conditions:

1. The player’s profile information is successfully updated in the system.
2. The player sees the updated information on their profile page.

Exceptions:

1. If the user enters invalid data, the system prompts an error message.

Priority: Medium

When available: Always available from “Manage Profile”.

Frequency of use: Rare

Channel to actor: The games GUI

Use case: Random (Opponent Selection)

Iteration: 1

Primary Actor: Player

Goal in context: Allow the player to match with a random opponent for a game.

Preconditions:

1. The player must be logged into the system
2. The player must be in a game lobby or on a game selection screen that supports multiplayer interactions.

Trigger: The player selects the "Random" option for opponent selection.

Scenario:

1. The player chooses a game they wish to play and selects the "Random Opponent" option.
2. The system accesses the matchmaking service to find another player who is also looking for a random match in the same game.
3. The system pairs the player with a randomly selected opponent who meets the matchmaking criteria.
4. The game session starts automatically once the opponent is found.

Post conditions:

1. The player is matched with a random opponent.
2. The game session between the matched players begins.

Exceptions:

1. If no opponents are available, the system informs the player and asks if they wish to wait.
2. If the matchmaking service encounters an error, the player is notified of the issue and returned to the previous menu.

Priority: Medium

When available: Always Available

Frequency of use: Frequent

Channel to actor: The Games GUI